

Django Real-time Chat Application

Este é um aplicativo de chat web em tempo real monolítico construído com Django e Channels, conforme as especificações fornecidas.

Configuração e Execução

Siga os passos abaixo para configurar e executar o projeto:

1. Pré-requisitos

Certifique-se de ter o Python 3.x e o pip instalados em seu sistema.

2. Clone o Repositório (ou descompacte o arquivo zip)

Se você recebeu um arquivo zip, descompacte-o em um diretório de sua escolha. Se estiver usando Git, clone o repositório:

Bash

```
git clone <URL_DO_REPOSITORIO>
cd django_chat_app
```

3. Crie e Ative um Ambiente Virtual

É altamente recomendável usar um ambiente virtual para gerenciar as dependências do projeto:

Bash

```
python -m venv venv
source venv/bin/activate # No Linux/macOS
# ou
venv\Scripts\activate   # No Windows
```

4. Instale as Dependências

Com o ambiente virtual ativado, instale as dependências listadas no requirements.txt :

Bash

```
pip install -r requirements.txt
```

5. Migrações do Banco de Dados

Aplique as migrações do banco de dados para criar as tabelas necessárias:

Bash

```
python manage.py makemigrations
python manage.py migrate
```

6. Crie um Superusuário (Opcional, para acessar o Admin)

Para acessar o painel de administração do Django, você pode criar um superusuário:

Bash

```
python manage.py createsuperuser
# Siga as instruções para criar o usuário e senha
```

7. Execute o Servidor de Desenvolvimento

Para iniciar o aplicativo, você precisará executar o servidor ASGI (Daphne, que já está incluído nas dependências):

Bash

```
daphne -b 0.0.0.0 -p 8000 chat_project.asgi:application
```

Se você quiser rodar o servidor Django padrão para testar as views HTTP (sem WebSockets):

Bash

```
python manage.py runserver
```

8. Acesse o Aplicativo

Abra seu navegador e acesse:

- **Página inicial do chat:** `http://127.0.0.1:8000/chat/`
- **Página de registro:** `http://127.0.0.1:8000/chat/register/`
- **Página de login:** `http://127.0.0.1:8000/chat/login/`
- **Painel de administração:** `http://127.0.0.1:8000/admin/` (use as credenciais do superusuário)

Estrutura do Projeto

- `chat_project/` : O diretório principal do projeto Django.
 - `settings.py` : Configurações do projeto, incluindo `INSTALLED_APPS` , `ASGI_APPLICATION` , `CHANNEL_LAYERS` , etc.
 - `urls.py` : URLs globais do projeto.
 - `asgi.py` : Configuração ASGI para lidar com WebSockets.
- `chat/` : O aplicativo Django principal para o chat.
 - `models.py` : Definição dos modelos de banco de dados (User, ChatRoom, Message, Membership).
 - `views.py` : Lógica para as views HTTP (registro, login, index, salas de chat, criação de chat).
 - `urls.py` : URLs específicas do aplicativo `chat` .
 - `consumers.py` : Lógica para lidar com as conexões WebSocket e mensagens em tempo real.
 - `routing.py` : Define o roteamento de WebSockets para os consumers.
- `templates/` : Contém os templates HTML do Django para a interface do usuário.
 - `chat/base.html` : Template base com Bootstrap para layout e estilos.
 - `chat/index.html` : Página inicial do chat com lista de chats e opções para criar novos.
 - `chat/login.html` : Página de login.
 - `chat/register.html` : Página de registro.
 - `chat/room.html` : Página da sala de chat com histórico de mensagens e entrada de texto.
 - `chat/search_users.html` : Página para buscar usuários.
- `requirements.txt` : Lista de todas as dependências Python do projeto.
- `db.sqlite3` : Arquivo de banco de dados SQLite (gerado após as migrações).

Funcionalidades Implementadas

- **Autenticação de Usuário:** Registro, login e logout usando o sistema de autenticação do Django.
- **Chats Privados:** Usuários podem iniciar conversas um-a-um.
- **Chats em Grupo:** Usuários podem criar grupos e adicionar membros.

- **Funcionalidade em Tempo Real:** Mensagens aparecem instantaneamente sem recarregar a página, usando Django Channels.
- **Histórico de Mensagens:** Carrega as últimas 50 mensagens ao entrar em uma sala de chat.
- **Interface do Usuário:** Construída com templates Django e Bootstrap para um design responsivo.

Notas Importantes

- **SQLite:** O projeto usa SQLite por padrão para simplicidade. Para PostgreSQL, você precisaria ajustar as configurações em `chat_project/settings.py`.
- **Eventlet/Socket.IO:** A especificação original mencionava `django-socketio` e `Eventlet`. No entanto, `django-socketio` é um projeto antigo e incompatível com versões recentes do Python/Django. A implementação foi feita com `Django Channels`, que é a solução recomendada e moderna para WebSockets no Django.
- **Testes:** Testes unitários básicos não foram incluídos devido ao escopo, mas são recomendados para um projeto de produção.
- **Escalabilidade:** Para aplicações de grande escala, um backend de channel layer como Redis seria necessário em vez do `InMemoryChannelLayer` atual.

Qualquer dúvida ou problema, sinta-se à vontade para perguntar!